

Installation et prise en main

Module 2 — Configuration de l'environnement Spark / PySpark

Objectifs pédagogiques

- Installer et configurer un environnement PySpark fonctionnel en local
- Utiliser les outils en ligne de commande de Spark (pyspark, spark-submit, spark-shell)
- Créer et configurer une SparkSession adaptée à différents contextes
- Naviguer dans la Spark UI pour monitorer et diagnostiquer une application
- Exécuter ses premiers programmes PySpark dans un notebook Jupyter

1. Pré-requis système

Prérequis système — Java & Python

Spark tourne sur la JVM — Java est obligatoire même pour PySpark

Java (JVM)

- Spark 3.x supporte Java 8, 11 et 17
- Java 11 recommandé pour sa stabilité
- Vérifier : `java -version`
- Linux/macOS : installer via SDKMAN
- Windows : JDK depuis adoptium.net + `JAVA_HOME`

Python

- PySpark supporte Python 3.8 à 3.12
- Vérifier : `python --version`
- Toujours utiliser un environnement virtuel
- `python -m venv .venv-spark`
- `source .venv-spark/bin/activate`

Bonne pratique : définir les variables d'environnement dans `~/.bashrc` ou `~/.zshrc` — `JAVA_HOME`, `SPARK_HOME`, `PYSPARK_PYTHON`, `PATH`

2. Installation de PySpark

Méthode 1 — via pip

Méthode recommandée pour le développement local — rapide et sans configuration manuelle

```
# Installation simple  
pip install pyspark==3.5.0  
  
# Avec les dépendances utiles pour le développement  
pip install pyspark==3.5.0 jupyter findspark
```

✓ **Avantage** : rapide, aucune configuration manuelle de Spark

⚠ **Limite** : moins flexible pour des configurations avancées de cluster

Méthode 2 — Installation manuelle (contrôle fin sur le cluster)

```
wget https://downloads.apache.org/spark/spark-3.5.0/spark-3.5.0-bin-hadoop3.tgz  
tar -xvzf spark-3.5.0-bin-hadoop3.tgz && mv spark-3.5.0-bin-hadoop3 /opt/spark-3.5.0  
export SPARK_HOME=/opt/spark-3.5.0 && pip install pyspark==3.5.0
```

Méthode 3 — Environnements cloud & vérification

Environnement	Avantage	Accès
Google Colab	Gratuit, PySpark installable en 1 commande	colab.research.google.com
Databricks Community Ed.	Cluster Spark managé + notebooks intégrés	community.cloud.databricks.com
Amazon EMR	Cluster AWS avec intégration S3 native	aws.amazon.com/emr
Azure HDInsight	Cluster Azure avec Spark préinstallé	azure.microsoft.com

Vérification de l'installation

```
# Vérifier la version PySpark
python -c "import pyspark; print(pyspark.__version__)"    # → 3.5.0

# Vérifier que la SparkSession se crée sans erreur
python -c "from pyspark.sql import SparkSession; \

spark = SparkSession.builder.master('local').appName('test').getOrCreate(); \

print('Spark', spark.version); spark.stop()"
```

3. La ligne de commande de Spark

Shells interactifs — pyspark et spark-shell

pyspark — REPL Python interactif

```
pyspark --master local[*]  
pyspark --master local[2] --driver-memory 2g --executor-memory 4g  
# spark et sc disponibles d'emblée dans le shell  
>>> df = spark.range(10) ; df.show()
```

Variables pré-initialisées : `spark` (SparkSession) et `sc` (SparkContext) — disponibles dès l'ouverture du shell

spark-shell — REPL Scala (utile pour lire la doc officielle)

```
spark-shell --master local[*]  
// Scala  
scala> val df = spark.range(10) ; df.show()
```

spark-submit — Soumission de jobs en production

L'équivalent de python mon_script.py mais pour Spark sur cluster

```
spark-submit \  
  --master <url-du-cluster>  --deploy-mode <client|cluster>  --num-executors 10 \  
  --executor-memory 8g  --executor-cores 4  --driver-memory 4g \  
  --conf spark.sql.shuffle.partitions=200 \  
  mon_script.py  [arguments...]
```

deploy-mode	Driver tourne sur...	Usage typique
client (défaut)	La machine qui lance spark-submit	Développement, débogage — logs visibles directement
cluster	Un nœud du cluster	Production — le terminal peut être fermé

```
# Développement local  
spark-submit --master local[*] mon_script.py  
  
# Production YARN  
spark-submit --master yarn --deploy-mode cluster --num-executors 10 --executor-memory 8g pipeline_etl.py
```

Récapitulatif des outils CLI

Quatre commandes, quatre usages bien distincts

pyspark

Python

Exploration interactive, développement et débogage rapide

spark-shell

Scala

Exploration interactive, lecture de la documentation officielle

spark-submit

Python/Scala/R

Exécution de jobs en production sur cluster

spark-sql

SQL

Requêtes SQL ad hoc directement en ligne de commande

4. La session Spark

SparkSession : point d'entrée unique

Création d'une SparkSession

Depuis Spark 2.0 : remplace et unifie SparkContext, SQLContext et HiveContext

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("NomDeLApplication") \
    .master("local[*]") \
    .config("spark.sql.shuffle.partitions", "8") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.driver.memory", "2g") \
    .getOrCreate() # crée ou récupère une session existante
```

💡 **getOrCreate()** : si une session existe déjà (Databricks, notebook partagé), elle est réutilisée. Ne jamais appeler `spark.stop()` entre chaque cellule d'un notebook.

- `appName` : nom visible dans la Spark UI — toujours choisir un nom explicite
- `shuffle.partitions` : défaut 200 (trop élevé en local — préférer 4 à 16 selon les cœurs)
- `adaptive.enabled` : AQE (Adaptive Query Execution) — Spark 3.x, active le repartitionnement dynamique

URL de master et accès au SparkContext

URL de master les plus courantes

```
# 1 thread (pas de parallélisme)
.master("local")

# N threads
.master("local[4]")    # 4 threads
.master("local[*]")    # tous les cœurs

# Cluster Standalone
.master("spark://192.168.1.10:7077")

# YARN
.master("yarn")

# Kubernetes
.master("k8s://https://k8s-api:6443")
```

Accès au SparkContext (bas niveau)

```
sc = spark.sparkContext
sc.version
sc.master
sc.defaultParallelism
sc.appName
```

Fermeture de session

```
# Fin de script : libérer les
ressources

spark.stop()
```

Configurations avancées : définir dans `$SPARK_HOME/conf/spark-defaults.conf` pour qu'elles s'appliquent à toutes les applications sans les coder en dur.

5. Spark UI

Interface de monitoring

Spark UI — votre outil de diagnostic

Interface web automatiquement disponible dès qu'une application Spark est lancée

Contexte	URL par défaut
Application en cours d'exécution	http://localhost:4040
Plusieurs applications simultanées	4040, 4041, 4042...
Spark History Server (jobs terminés)	http://localhost:18080

Jobs

Jobs déclenchés par les actions —
durée, stages, statut

Stages

Tâches lentes (stragglers), shuffle
read/write, durées

Storage

RDD/DataFrames mis en cache —
mémoire utilisée vs disponible

Executors

Ressources par executor — détecter le
data skew

SQL/DataFrame

Plans d'exécution logiques et
physiques — optimisation

Lire un plan d'exécution — explain()

Disponible dans le terminal ET dans l'onglet SQL de la Spark UI

```
df_filtre = df.filter(df["montant"] > 1000).groupBy("region").sum("montant")

df_filtre.explain()           # plan physique
df_filtre.explain(mode="extended") # plan + optimisations Catalyst
```

Sortie typique d'explain() :

```
== Physical Plan ==
*(2) HashAggregate(keys=[region], functions=[sum(montant)])
+- Exchange hashpartitioning(region, 8)  ← shuffle réseau
    +- *(1) HashAggregate(keys=[region], functions=[partial_sum(montant)])
        +- *(1) Filter (montant > 1000)
            +- FileScan csv [region, montant]
```

- Lire de bas en haut : FileScan → Filter → HashAggregate partial → Exchange (shuffle) → HashAggregate final
- Exchange = shuffle réseau = étape la plus coûteuse — à minimiser dans les pipelines de production

6. Spark et Jupyter Notebook

Jupyter — Configuration et bonnes pratiques

Configuration initiale

```
import findspark
findspark.init() # localise SPARK_HOME

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Notebook") \
    .master("local[*]") \
    .config("spark.sql.shuffle.partitions","4") \
    .getOrCreate()
```

Bonnes pratiques

```
# ✅ Toujours limiter
df.show(20)
df.limit(100).show()

# ✅ toPandas sur petits sous-ensembles
df.filter(...).limit(1000).toPandas()

# ❌ Ne jamais faire
df.toPandas() # sature la RAM !
df.collect()  # même problème
```

Lancer Jupyter directement depuis pyspark :

```
export PYSPARK_DRIVER_PYTHON=jupyter
export PYSPARK_DRIVER_PYTHON_OPTS="notebook --no-browser --port=8888"
pyspark --master local[*] # ouvre directement Jupyter avec spark + sc pré-initialisés
```

Premier programme complet

```
from pyspark.sql import SparkSession, functions as F

spark =
SparkSession.builder.appName("AnalyseVentes").master("local[*]").config("spark.sql.shuffle.partitions"
, "4").getOrCreate()

data = [("Alice", "Paris", 1500.0), ("Bob", "Lyon", 800.0), ("Claire", "Paris", 2200.0), ...]

df = spark.createDataFrame(data, schema=["nom", "ville", "ca"])

df.filter(F.col("ca") > 1000) \
    .withColumn("ca_ht", F.col("ca") / 1.2) \
    .groupBy("ville").agg(F.sum("ca"), F.count("*"), F.avg("ca")) \
    .orderBy(F.col("ca").desc()).show()

spark.stop()
```

Résultat : Paris = 6 700 € (3 commerciaux) · Lyon = 1 100 € (1 commercial) — filtrage + agrégation + tri en un seul pipeline

Résumé du module

Concept	Points clés à retenir
Prérequis	Java (JVM) obligatoire même pour PySpark — Python 3.8+ recommandé
pip install	pip install pyspark — méthode la plus simple en développement local
pyspark	Shell interactif Python — spark et sc disponibles d'emblée
spark-submit	Outil de production — soumission de jobs sur cluster (client vs cluster mode)
SparkSession	Point d'entrée unique depuis Spark 2.x — toujours via .builder.getOrCreate()
local[*]	Mode développement — utilise tous les cœurs de la machine locale
Spark UI	localhost:4040 — indispensable pour le monitoring et le diagnostic
Notebooks	Intégration naturelle avec Jupyter — toujours limiter toPandas() aux petits volumes

Exercices

Ex. 1 — Installation et vérification *(20 min)* Installer PySpark en venv, vérifier via shell Python, lancer `pyspark --master local[*]`, créer `spark.range(100)` et observer le job dans la Spark UI.

Ex. 2 — spark-submit *(20 min)* Écrire le programme `AnalyseVentes` dans `analyse_ventes.py`, l'exécuter avec `spark-submit --master local[*]`, identifier jobs et stages dans la Spark UI.

Ex. 3 — Configuration et plan d'exécution *(30 min)* Créer une `SparkSession` avec `shuffle.partitions=4`, lire un CSV, faire un `groupBy + count()`, appeler `.explain(mode='extended')` et identifier `FileScan`, `HashAggregate` et `Exchange (shuffle)`.

Ex. 4 — Notebook Jupyter *(30 min)* Lancer Jupyter avec PySpark, reproduire `AnalyseVentes` en notebook, afficher le résultat final avec `.toPandas()` et documenter chaque cellule avec du Markdown.

Pour aller plus loin

Documentation & Ressources

- Configuration Spark : spark.apache.org/docs/latest/configuration.html
- spark-submit : spark.apache.org/docs/latest/submitting-applications.html
- Databricks Community Edition : cluster Spark managé gratuit — idéal pour les TP
- Spark UI expliquée : youtu.be/BVMIT7WO-5I (Databricks)

Fichier de configuration global

```
# $SPARK_HOME/conf/spark-defaults.conf — appliqué à toutes les applications
spark.sql.shuffle.partitions      8
spark.sql.adaptive.enabled       true
spark.driver.memory              2g
```

Module suivant → Module 3 : Les structures de données Spark — RDD, DataFrame, Dataset et leurs différences